

Aadithya Sivasankar N R's Portfolio Documentation

Comprehensive Technical Architecture, Languages, Frameworks & Robot Animation Deep-Dive

1. Technology Stack & Languages Used

This modern, responsive portfolio application is built with a zero-bloat, high-performance architecture using modern web standards and APIs:

- 🔗 **HTML5 (HyperText Markup Language):** Semantic page structure, accessibility attributes, navigation landmarks, meta tags, and deep SVG embedding.
- 🔗 **CSS3 (Cascading Style Sheets):** Vanilla CSS design system using Custom Properties (Variables), 3D perspective transforms, Flexbox/Grid layouts, Glassmorphism backdrop-filters, and keyframe animations.
- 🔗 **JavaScript (Vanilla ES6+):** Pointer events listener, dynamic vector coordinate math, linear interpolation (Lerp) physics loop on requestAnimationFrame, and asynchronous fetch API.
- 🔗 **Scalable Vector Graphics (SVG):** Fully customized, resolution-independent vector illustrations with gradient shaders, glow filters (feGaussianBlur), and clipping paths (clipPath).
- 🔗 **Node.js Backend Server (server.js):** Lightweight built-in HTTP server providing MIME-type static file serving and port-collision auto recovery.
- 🔗 **FormSubmit API (Cloud Forwarder):** Asynchronous REST endpoint handling contact form submissions and forwarding them directly to your Gmail without server backend overhead.
- 🔗 **Font Awesome 6 & Google Fonts (Outfit):** Iconography suite for navigation/social links, combined with modern geometric typography.

2. System Architecture & File Organization

The codebase follows a modular separation of concerns between structure, visual presentation, and interactive mechanics:

- 🔗 **index.html:** Main document containing Navbar, Hero Section with Cyber Robot SVG, About (Education & Internships), Technical Skills, Projects, Contact Form, and Footer.
- 🔗 **style.css:** Design system tokens, color palettes, glassmorphism cards, responsive media queries (desktop, tablet, mobile), and 3D perspective canvas rules.
- 🔗 **script.js:** Core logic: Mobile hamburger menu toggle, navbar scroll effects, scroll-spy navigation, asynchronous contact form dispatcher, and robot mouse tracking controller.
- 🔗 **server.js:** Zero-dependency Node HTTP web server running on port 3000.

3. Deep-Dive: How the Robot Cursor-Tracking Animation Works

The interactive robot character in the Hero section uses a multi-tiered pipeline involving Vector SVG layering, CSS 3D perspectives, and Javascript vector trigonometry with Linear Interpolation (Lerp).

Phase A: Layered SVG Construction (index.html)

The robot is constructed in vector layers so that individual parts can be isolated and transformed independently in 2D and 3D space:

- 🔗 **#char-head:** The entire head group (purple hood, silver rim, cyan headphones with pulsing LED antenna, and visor). Rotates in 3D (pitch, yaw, roll).
- 🔗 **#char-visor-group:** Glossy dark shield helmet screen with specular glass reflection arc.

â€¢#char-eyes-container (clip-path): Clipped strictly to the inside of the visor so the eyes never bleed outside the helmet screen boundary.

â€¢#char-eyes: Dual glowing cyan-white circular eyes with radial gradient shaders and feGaussianBlur glow filters.

Phase B: 3D Canvas & Perspective Setup (style.css)

In style.css, the parent container is given a 3D perspective to enable genuine depth:

CSS: 3D Perspective and Transform Origins

```
.hero-visual {
  perspective: 1200px;
  transform-style: preserve-3d;
}
.char-head-group {
  transform-origin: 205px 175px;
  will-change: transform;
  transition: transform 0.06s cubic-bezier(0.2, 0.8, 0.2, 1);
}
.char-eyes-group {
  transform-origin: 205px 172px;
  will-change: transform;
}
```

Phase C: Mouse Vector Math & Normalized Offsets (script.js)

When the user moves their mouse (or touches their mobile screen), script.js computes the relative vector from the character head center:

JavaScript: Mouse Vector & Normalization Math

```
const rect = characterContainer.getBoundingClientRect();
const charCenterX = rect.left + rect.width * 0.49;
const charCenterY = rect.top + rect.height * 0.33;

const deltaX = clientX - charCenterX;
const deltaY = clientY - charCenterY;

// Normalize between -1.0 and +1.0
const normX = Math.max(-1, Math.min(1, deltaX / (window.innerWidth * 0.4)));
const normY = Math.max(-1, Math.min(1, deltaY / (window.innerHeight * 0.4)));

targetEyeX = normX * 24; // Eye displacement in px
targetEyeY = normY * 18;
targetHeadRotY = normX * 18; // Head Yaw in degrees
targetHeadRotX = -normY * 14; // Head Pitch in degrees
```

Phase D: Smooth Physics via Linear Interpolation (Lerp)

Rather than jumping instantly to the mouse position, script.js runs a 60 FPS requestAnimationFrame loop that smoothly glides (damps) towards the target position:

JavaScript: Lerp Animation Loop

```
function animateCharacter() {
  const lerpEye = 0.10;
  const lerpHead = 0.08;

  currentEyeX += (targetEyeX - currentEyeX) * lerpEye;
  currentEyeY += (targetEyeY - currentEyeY) * lerpEye;
  currentHeadRotY += (targetHeadRotY - currentHeadRotY) * lerpHead;
  currentHeadRotX += (targetHeadRotX - currentHeadRotX) * lerpHead;

  // Apply transforms directly to SVG elements
  charEyes.style.transform = `translate(${currentEyeX.toFixed(2)}px, ${currentEyeY.toFixed(2)}px)`;
  charHead.style.transform = `translate(${currentHeadTransX}px, ${currentHeadTransY}px) rotateY(${currentHeadRotY})`;

  requestAnimationFrame(animateCharacter);
}
```

Phase E: Micro-Interactions & Easter Eggs

- ⚡ Periodic Blinking (triggerRandomBlink): A randomized timer triggers every 3â 6.5 seconds, temporarily scaling scaleY of the eyes to 0.08 for 160ms.
- ⚡ Rest Position Decay (mouseleave): When the mouse leaves the browser window, targets smoothly return to zero, centering the character face.
- ⚡ Celebration Click Effect: Clicking the robot triggers a bounce keyframe animation (robotCheer) and a celebratory double-blink.

4. Real-Time Gmail Dispatcher (Get In Touch Section)

The contact form in index.html is linked to FormSubmit API with an AJAX listener in script.js:

- ⚡ Asynchronous Fetch: Submits JSON data containing sender name, email, subject, and message without triggering page reloads.
- ⚡ Visual Feedback: Changes button text from "Sending..." to "Message Sent! â " in emerald green upon successful delivery.
- ⚡ Fail-Safe Fallback: If the visitor is offline or a browser blocker interferes, it automatically redirects to the user default email client (mailto:).

Key Project Highlights

This portfolio combines crisp vector graphics, zero-dependency Node serving, GPU-accelerated 3D transforms, real-time physics interpolation, and cloud-forwarded Gmail delivery to deliver an unforgettable user experience.